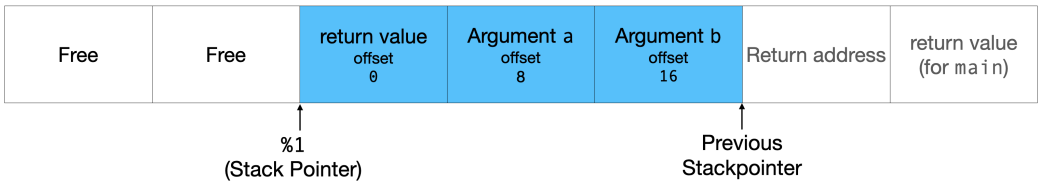
## Learning Objectives

- Understand the role of the frame pointer and how it differs from the stack pointer
- Understand the stack layout for function parameters, local variables, and return values
- Distinguish between arguments passed by the caller and parameters accessed by the callee
- Extend the code generator to implement the new calling convention
- Use the symbol table to manage stack offsets for parameters and local variables
- Generate code for `return` statements using a shared function epilogue

## Revisiting Our Calling Convention from Worksheet 14

In Worksheet 14, we developed a calling convention for passing values to functions and returning results. Let's take another look at the stack layout for a call to `diff(72, 30)`.
First, a small distinction in terminology: **arguments** are the values supplied by the caller, whereas **parameters** are the variables through which the callee accesses these values. In our example, `72` and `30` are arguments, while `a` and `b` are parameters of `diff`.

### Before the function call (caller's perspective):




The caller has reserved space for the return value and the arguments. Relative to the current stack pointer `%1`, their offsets are:

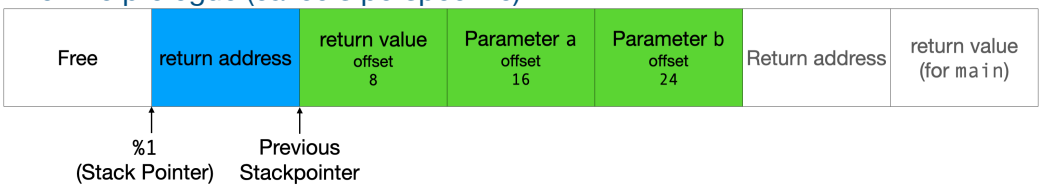
- Return value: `0(%1)`
- Argument `a`: `8(%1)`
- Argument `b`: `16(%1)`

The notation `offset(%register)` follows the addressing syntax used by our assembly instructions. For example:

```
movq    8(%1), %4     # fetch from memory into %4
movq    %4, 8(%1)     # store %4 into memory
```

Here, in both cases, the memory address is computed by adding the offset (8 bytes) to the value stored in register `%1`.

### After the prologue (callee's perspective):




The callee has reserved 8 bytes on the stack and saved the return address there. Consequently, the stack pointer has changed, and so have the offsets:

- Return value: `8(%1)`
- Parameter `a`: `16(%1)`
- Parameter `b`: `24(%1)`

Notice that the values are still stored at exactly the same memory addresses. Only the stack pointer has moved.
Now imagine that the callee could still access the **caller's stack pointer**, i.e., its value immediately before the call. The offsets for accessing parameters inside the function would then be identical to those used by the caller for passing arguments.
This would greatly simplify our compiler: we could store these offsets in the symbol table and reuse them both when passing arguments and when accessing parameters.

### The Basic Idea Behind Our New Calling Convention: Stack Pointer and Frame Pointer

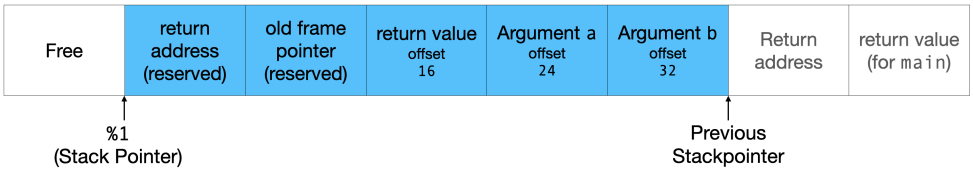
That is precisely the idea behind introducing a frame pointer: a stable reference point for accessing parameters and local variables. We will use register `%2` as our *frame pointer*. Since the caller may already be using this register, we must save its previous value on the stack and restore it before returning.

## Our New Stack Layout

Let's revisit our call to `diff(72, 30)`, to introduce our new calling convention.

### Before the function call (caller's perspective):

The caller now reserves space for two additional entries: the return address and the old frame pointer. The stack layout looks as follows:

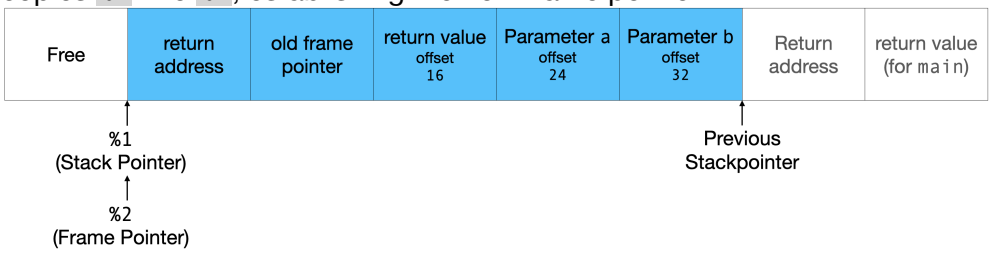



Relative to the stack pointer `%1`, the offsets are:

- Return address: `0(%1)`
- Old frame pointer: `8(%1)`
- Return value: `16(%1)`
- Argument `a`: `24(%1)`
- Argument `b`: `32(%1)`

### After the prologue (callee's perspective)

The callee stores the return address and the old frame pointer in the reserved locations. It then copies `%1` into `%2`, establishing the new frame pointer.



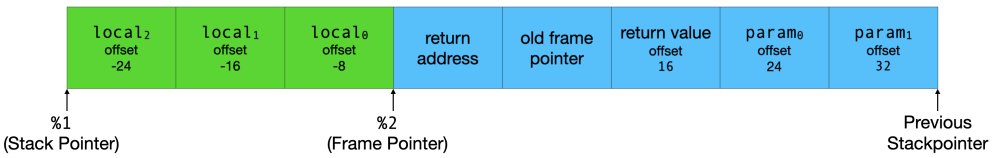

The prologue does not change the stack pointer, and the frame pointer now points to the same location. Consequently, the offsets relative to the frame pointer are identical to those previously used by the caller relative to the stack pointer:

- Return address: `0(%2)`
- Old frame pointer: `8(%2)`
- Return value: `16(%2)`
- Parameter `a`: `24(%2)`
- Parameter `b`: `32(%2)`

The important observation is that the caller uses offsets relative to the stack pointer, while the callee uses the same offsets relative to the frame pointer.
This will become particularly useful when we introduce local variables: the stack pointer may change during the execution of a function, but the frame pointer remains fixed.

## Looking Ahead: Local Variables

Why do we need two pointers if `%1` and `%2` initially point to the same location? The stack pointer `%1` may change, for example when the function calls another function. The frame pointer `%2`, however, remains fixed, allowing parameters to be accessed using constant offsets—regardless of changes to the stack pointer.
Later in this worksheet, we will also reserve stack space for local variables. This gives us the following stack layout:




Notice the simple pattern:

- **Parameters** have fixed, positive offsets relative to `%2`.
- **Local variables** have fixed, negative offsets relative to `%2`.

This is exactly what we need for our compiler: the symbol table can store a fixed offset for each parameter or local variable, and the code generator can use it directly whenever that variable is accessed.

### Step 1 — Putting It All Together (Again!)

In Worksheet 14, we implemented the following program in ULM assembly:

```
fn diff(a: u64, b: u64): u64
{
    return a - b;
}

fn main(): u64
{
    return diff(72, 30);
}
```

Let's revisit this example using our new calling convention. The program should still terminate with exit code `42`, but this time we will use `%2` as the frame pointer.

### Tasks

1. **Calling main.** Create a new assembly file `ex1.s`. Start with the following startup code:

```
.text
    loadz   0,      %1

    subq    24,     %1,     %1
    call    main,   %3
    movq    16(%1), %4
    addq    24,     %1,     %1
    halt    %4
```

   Notice that we now reserve 24 bytes: 8 bytes each for the return address, the old frame pointer, and the return value. Also notice that the startup code does not initialize %2. Its initial value is undefined, but this is perfectly fine because the startup code never uses it.

2. **Defining `diff`.** Next, add the following implementation of `diff`:

```
diff:
    movq    %3,     (%1)        # store return address
    movq    %2,     8(%1)       # save previous frame pointer
    movq    %1,     %2          # set new frame pointer

    // begin of function body
    movq    24(%2), %4          # fetch a
    movq    32(%2), %5          # fetch b
    subq    %5,     %4,     %4  # subtract b from a
    movq    %4,     16(%2)      # store %4 as return value
    // end of function body

    movq    %2,     %1          # restore stack pointer at function entry
    movq    8(%1),  %2          # restore previous frame pointer
    movq    (%1),   %3          # restore return address
    ret     %3
```

   The prologue saves the return address and the old frame pointer, then establishes `%2` as the new frame pointer. Unlike in Worksheet 14, the stack pointer does not need to be adjusted: the caller has already reserved the necessary space. The function body accesses both parameters using fixed offsets relative to `%2` and stores the result at `16(%2)`.
   The epilogue first restores `%1` to its value at function entry. It then restores the previous frame pointer and the return address. This works even if the stack pointer has changed during the execution of the function.

3. **Defining `main`.** Now add the following implementation of `main`:

```
main:
    movq    %3,     (%1)        # store return address
    movq    %2,     8(%1)       # save previous frame pointer
    movq    %1,     %2          # set new frame pointer
    /* function body here */
    movq    %2,     %1          # restore stack pointer at function entry
    movq    8(%1),  %2          # restore previous frame pointer
    movq    (%1),   %3          # restore return address
    ret     %3
```

   Notice that `main` uses exactly the same prologue and epilogue as `diff`. These patterns do not depend on the number of parameters or on the function body.

4. **Calling `diff` with two arguments.** Replace `/* function body here */` with:

```
    subq    40,     %1,     %1
    loadz   72,     %4
    movq    %4,     24(%1)
    loadz   30,     %4
    movq    %4,     32(%1)
    call    diff,   %3
    movq    16(%1), %4
    addq    40,     %1,     %1

    movq    %4,     16(%2)          # store %4 as return value
```

   The caller reserves 40 bytes: 16 bytes for the return address and old frame pointer, 8 bytes for the return value, and 16 bytes for the two arguments.
   Notice that the arguments are stored at offsets `24(%1)` and `32(%1)`, exactly where `diff` accesses them relative to `%2`. After the call, `main` retrieves the result from `16(%1)` and stores it in its own return-value slot at `16(%2)`.
5. **Assemble and run the program:**

```
ulmas -o ex1 ex1.s
ulm ex1
echo $?
```

   The exit code should be `42`.

### Looking Ahead: Generating `return` Statements

In both `diff` and `main`, we take advantage of the fact that the return value is stored immediately before the function's epilogue. We can simply let execution continue into the epilogue without an explicit jump.
Recall that in Worksheet 17, we used a little hack: We observed that our expression code generator leaves the final result in register `%4`. We therefore modified the epilogue to store `%4` as the function's return value.
But what would a proper implementation of a `return` statement look like?
The code generator should be able to take the value from any register, store it in the function's return-value slot, and jump to the epilogue. For example, the end of `diff` could look like this:

```
    subq    %5, %4, %4      # subtract b from a
    movq    %4, 16(%2)      # store return value
    jmp     .leave.diff     # jump to the epilogue

    // end of function body

.leave.diff:
    movq    %2, %1
    movq    8(%1), %2
    movq    (%1), %3
    ret     %3
```

In our handwritten assembly, we have effectively optimized away this jump because the epilogue follows immediately. Later in this worksheet, we will extend our code generator to handle the general case, where a `return` statement may appear anywhere in a function body.

### Step 2 — Making Room for Local Variables

Let's artificially extend our example from Step 1 by introducing a local variable `r`:

```
fn diff(a: u64, b: u64): u64
{
    local r: u64;
    r = a - b;
    return r;
}

fn main(): u64
{
    return diff(72, 30);
}
```

The result is still `42`, but now `diff` needs space for a local variable on the stack. In this step, we

will implement the `return` statement the way our code generator will later handle it: by storing the return value and jumping to a shared function epilogue. Although the jump is unnecessary in this particular example, we will deliberately keep it to establish a general pattern.

Once more, we will implement this program directly in ULM assembly to see what our code generator will eventually need to produce.

## Tasks

1. **Reserving space for a local variable.** Copy `ex1.s` to `ex2.s` and replace the implementation of `diff` with:

```
diff:
    movq    %3,     (%1)        # store return address
    movq    %2,     8(%1)       # store current frame pointer
    movq    %1,     %2          # set new frame pointer
    subq    8,      %1,     %1  # space for local variables

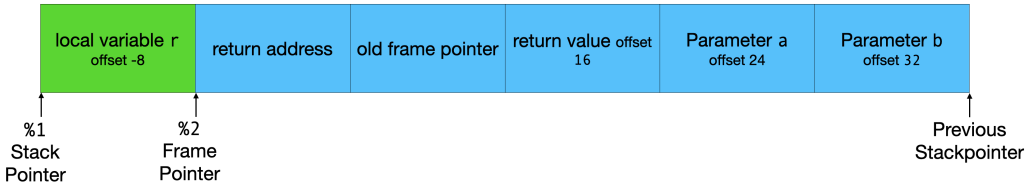
    // begin of function body
    movq    24(%2), %4          # fetch a
    movq    32(%2), %5          # fetch b
    subq    %5,     %4,     %4  # subtract b from a
    movq    %4,     -8(%2)      # store result in local variable
    movq    -8(%2), %4          # return local variable
    movq    %4,     16(%2)
    jmp     .leave.diff
    // end of function body

.leave.diff:
    movq    %2,     %1          # restore previous stack pointer
    movq    8(%1),  %2          # restore previous frame pointer
    movq    (%1),   %3          # restore return address
    ret     %3
```

After establishing the frame pointer, the additional instruction

```
    subq    8,      %1,     %1  # space for local variables
```

reserves 8 bytes on the stack for our local variable `r`. The stack now looks as follows:




Notice how the function body implements our modified program:

```
    movq    24(%2), %4
    movq    32(%2), %5
    subq    %5,     %4,     %4
    movq    %4,     -8(%2)      # store result in local variable
```

First, the parameters `a` and `b` are loaded, and `b` is subtracted from `a`. The result is then stored in the stack slot reserved for `r`, at offset `-8(%2)`.

Next, we retrieve `r` and store its value in the return-value slot and jump to the epilogue:

```
    movq    -8(%2), %4          # return local variable
    movq    %4,     16(%2)
    jmp     .leave.diff
```

Of course, this is rather wasteful: we load `r` back into the very register that already held its value! An optimizing compiler could eliminate these unnecessary memory accesses, giving us essentially the same assembly code as in Step 1.

But optimization is not our goal here. We want a simple, systematic way to generate correct code for local variables.

Reassemble and run `ex2.s`. The exit code should still be `42`.

2. **Defining the stack size later.** Copy `ex2.s` to `ex3.s`. We will now make two small changes to function `diff`:
   (i) Replace

```
subq    8,      %1,     %1  # space for local variables
```

   with

```
subq    .loc0,  %1,     %1  # space for local variables
```

   (ii) and add the following directive immediately **after** the function:

```
.equ    .loc0,  8           # space for local variables
```

   Notice that `.loc0` is not yet defined when the assembler encounters the `subq` instruction. This is perfectly fine: the assembler can resolve references to symbols defined later in the assembly code.
   The `.equ` directive defines `.loc0` to be `8` without generating an instruction.

   Reassemble and run `ex3.s`. The exit code should still be `42`.

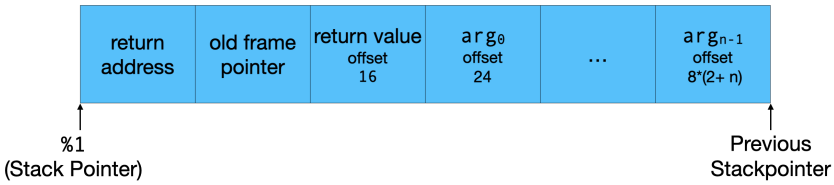
   ### Why is this useful for our compiler?
   Our parser generates assembly code while parsing a function. When emitting the prologue, it does not yet know how much stack space the local variables will require. We solve this by using a unique symbol for each function (`.loc0`, `.loc1`, ...), defining its value only after the function body has been parsed.

## Our Calling Convention at a Glance

Before updating our code generator, let's summarize the rules of our new calling convention.
We assume that every function has `n` arguments, `arg`$_0$, ..., `arg`$_{n-1}$, each occupying 8 bytes, and an 8-byte return value.

### Caller

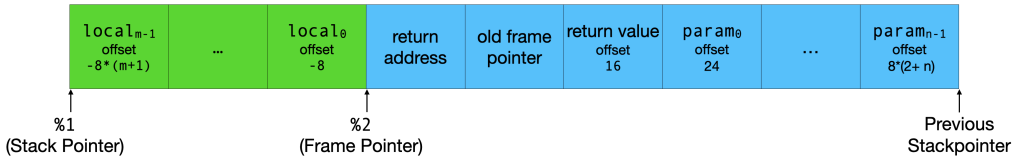
The caller accesses arguments and the return value using offsets relative to the stack pointer `%1`.




| What | Size / Memory location relative to `%1` |
|---|---|
| Stack space reserved | $8(n+3)$ bytes |
| Return value | `16(%1)` |
| Argument `arg`$_k$, $(0 \le k < n)$ | `8 * (k + 3)(%1)` |

### Callee

The callee accesses parameters, the return value, and its `m` local variables using offsets relative to the frame pointer `%2`.




| What | Size / Memory location relative to `%2` |
|---|---|
| Return value | `16(%2)` |
| Parameter `param`$_k$, $(0 \le k < n)$ | `8 * (k + 3)(%2)` |
| Local variable `local`$_k$, $(0 \le k < m)$ | `-8 * (k + 1)(%2)` |

### Prologue

Every function uses essentially the same prologue, regardless of its number of parameters or local variables:

```
    movq    %3,     (%1)
    movq    %2,     8(%1)
    movq    %1,     %2
    subq    .locID, %1,     %1
```

Here, `ID` is a unique number assigned to each function (`.loc0`, `.loc1`, ...). The symbol `.locID` is defined only after the function body, when the required stack space is known.

### Epilogue and Return Statements

Our code generator will give each function a unique *leave label*, using the same `ID` as for `.locID`:

```
.leaveID:
    movq    %2,     %1
    movq    8(%1),  %2
    movq    (%1),   %3
    ret     %3
```

This allows us to generate code for a `return` statement anywhere in the function body. We simply store the value of a register in the return-value slot and jump to the shared epilogue:

```
    movq %4, 16(%2)
    jmp  .leaveID
```

The `.locID` symbol is defined after the epilogue, using an `.equ` directive.

## Step 3 — Updating Our Code Generator: Function Calls

Before adding new features to our code generator, let's update it to use our new calling convention. Consider `xtest_gen_01.abc` from the `not-abc` repository:

```
@ "gen.hdr"

fn main()
{
    genInit();

    genFuncDefBegin("main");
    genCallBegin("getchar", 0);
    local ch: GenReg = genCallEnd();

    genCallBegin("putchar", 1);
    genCallAddArg(ch, 0);
    genReleaseReg(genCallEnd());
    genReleaseReg(ch);

    genFuncDefEnd();
}
```

This test program uses our code generator to produce a ULM assembly program that reads a character using `getchar` and writes it back using `putchar`. Several code generator functions used by this test program still implement our old calling convention. We will update them step by step in the following tasks. Most of the necessary changes follow directly from the rules summarized above. Some modifications will be given explicitly; the main goal is to revisit our calling convention and see how it translates into code generation.

**Before you start:** Build everything using `make`. Then generate, assemble, and execute a ULM program with:

```
./xtest_gen_01 > foo.s
ulmas -o foo foo.s
ulm foo
```

At this point, the program should still work: Type a character, and it should be echoed back. As we update individual code generator functions, the generated program will temporarily stop working correctly. **This is expected!** After each change, repeat the commands above and observe what happens.
Also inspect the generated assembly code using `cat foo.s`. Try to understand how each change to the code generator affects the assembly it produces.

## Tasks

1. **Updating `genFuncDefBegin()` and `genFuncDefEnd()`.** We start by updating the code generator functions responsible for generating function prologues and epilogues. First, introduce a few global variables to keep track of the current function definition:

```
global fnDefCount: int;
global paramCount: int = 0;
global localVarCount: int = 0;
global inFnDef: bool;
```

Replace `genFuncDefBegin()` and `genFuncDefEnd()` with:

```
fn genFuncDefBegin(name: ->readonly char)
{
    assert(!inFnDef);
    setSegment(TEXT);
    paramCount = 0;
    localVarCount = 0;
    inFnDef = true;
    printf("%s:\n", name);
    printf("\tmovq \t%%3, \t(%%1) \t\t# store return address\n");
    printf("\tmovq \t%%2, \t8(%%1) \t\t# save previous frame pointer\n");
    printf("\tmovq \t%%1, \t%%2 \t\t# set new frame pointer\n");
    printf("\tsubq \t.loc%d, \t%%1, \t%%1\n", fnDefCount);
    printf("\t// begin of function body\n");
}

fn genFuncDefEnd()
{
    assert(inFnDef);
    setSegment(TEXT);
    printf("\t// end of function body\n");
    printf(".leave%d:\n", fnDefCount);
    printf("\tmovq \t%%2, \t%%1 \t\t# restore stack pointer\n");
    printf("\tmovq \t8(%%1), \t%%2 \t\t# restore previous frame pointer\n");
    printf("\tmovq \t(%%1), \t%%3 \t\t# restore return address\n");
    printf("\tret \t%%3\n");
    printf("\t.equ \t.loc%d, \t%d\n", fnDefCount, 8 * localVarCount);
    ++fnDefCount;
    inFnDef = false;
}
```

Notice how we use a few global variables to maintain the state of our code generator. In a sense, we are implementing a small finite-state machine:

- `inFnDef` records whether we are currently generating a function definition. The assertions help us detect invalid transitions.
- `fnDefCount` gives each function unique labels (`.leave0`, `.loc0`, `.leave1`, `.loc1`, ...).
- `paramCount` and `localVarCount` keep track of the number of parameters and local variables in the current function.

When generating the function prologue, we do not yet know how many local variables the function will have. Nevertheless, we can already emit an instruction such as

```
subq .loc0, %1, %1
```

The symbol `.loc0` is defined only after the function body has been generated:

```
.equ .loc0, 0
```

Its value is determined by `localVarCount`. This allows us to generate assembly code in a single pass, without having to go back and modify the function prologue later.

2. **Updating `genCallBegin()`, `genCallAddArg()`, and `genCallEnd()`.** Next, we update the caller side of our calling convention. Recall that the caller must reserve space for the return address, the previous frame pointer, the return value, and all arguments. The arguments are stored in their natural order, starting at offset 24. We can keep our existing `CallInfo` structure and global variable:

```
struct CallInfo
{
    fnName:  ->readonly char;
    numArgs: unsigned;
};

global callInfo: CallInfo;
```

Update the three functions as follows:

```
fn genCallBegin(fnName: ->readonly char, numArgs: unsigned)
{
    setSegment(TEXT);
    assert(!callInfo.fnName);
    callInfo.fnName = fnName;
    callInfo.numArgs = numArgs;

    printf("\tsubq \t%u, \t%%1, \t%%1\n", 8*(callInfo.numArgs + 3));
}

fn genCallAddArg(argReg: GenReg, argPos: unsigned)
{
    setSegment(TEXT);
    printf("\tmovq \t%%%d, \t%u(%%1)\n", argReg, 8 * (argPos + 3));
}

fn genCallEnd(): GenReg
{
    setSegment(TEXT);
    local retVal: GenReg = getReg();
    printf("\tcall \t%s, %%3\n", callInfo.fnName);
    printf("\tmovq \t16(%%1), \t%%%u\n", retVal);
    printf("\taddq \t%u, \t%%1, \t%%1\n", 8 * (callInfo.numArgs + 3));
    callInfo.fnName = nullptr;
    return retVal;
}
```

Compare these implementations with the formulas in Our Calling Convention at a Glance:

- `genCallBegin()` reserves `8 * (numArgs + 3)` bytes on the stack.
- `genCallAddArg()` stores argument `argPos` at offset `8 * (argPos + 3)` relative to `%1`.
- `genCallEnd()` retrieves the return value from `16(%1)` and restores the stack pointer.

Notice that the argument offset no longer depends on the total number of arguments. This is a consequence of storing arguments in their natural order.

3. **Updating** `genInit()`**.** There is one more place where our old calling convention is still used: `genInit()`. Besides generating the startup code, this function also generates the implementations of `getchar` and `putchar`. These functions must follow our new calling convention as well. Replace `genInit()` with:

```
fn genInit()
{
    setSegment(TEXT);
    printf("\tloadz \t0, \t%%1\n\n");

    genCallBegin("main", 0);
    local exitCode: GenReg = genCallEnd();
    printf("\thalt \t%%%u\n\n", exitCode);
    genReleaseReg(exitCode);

    genFuncDefBegin("putchar");
    printf("\tmovq \t24(%%2), \t%%4\n");
    printf("\tputc \t%%4\n");
    genFuncDefEnd();

    genFuncDefBegin("getchar");
    printf("\tgetc \t%%4\n");
    printf("\tmovq \t%%4, \t16(%%2)\n");
    genFuncDefEnd();
}
```

Notice that we now use `genFuncDefBegin()` and `genFuncDefEnd()` to generate the prologues and epilogues of our built-in functions. However, we still generate a few assembly instructions directly using `printf()`. This is a small shortcut: Our code generator does not yet provide functions for every assembly instruction we need. Also notice that `getchar` and `putchar` access their return value and argument using the frame pointer `%2`, just like any other function following our calling convention.

**Test your changes:** Rebuild everything using `make`, regenerate `foo.s`, and inspect the assembly code. Then execute the program. You should now be able to enter a character and see it echoed back.

### Step 4 — Preparing for Variables and Return Statements

In Worksheet 20, we will extend our parser to support variables in expressions and `return` statements. First, we need to prepare our code generator. Remember the symbol table from Worksheet 9? Time to put it to use again!
Add these declarations to `gen.hdr`:

```
extern fn genFuncDefParam(ident: ~>readonly char): bool;
extern fn genAddLocal(ident: ~>readonly char): bool;
extern fn genReturn(val: GenReg);
```

Consider `xtest_gen_ex3.abc`:

```
@ "gen.hdr"
@ "symtab.hdr"

fn main()
{
    // fn diff(a, b) {
    genFuncDefBegin("diff");
    genFuncDefParam("a");
    genFuncDefParam("b");
    // local r: u64;
    genAddLocal("r");
    // print offsets (relative to frame pointer)
    symtabPrintLocals();
    genReturn(genConstant("42"));
    // }
    genFuncDefEnd();
}
```

The implementation is straightforward: We translate the callee offset formulas from *Our Calling Convention at a Glance* into code. Parameters and local variables are registered in the symbol table with their offsets relative to `%2`; `genReturn()` stores the return value and jumps to the function epilogue.
Add the following implementations to `gen.abc` (remember to include `symtab.hdr`):

```
fn genFuncDefParam(ident: ~>readonly char): bool
{
    assert(inFnDef);
    local added: bool = symtabAddLocal(ident, 8 * (paramCount + 3));
    ++paramCount;
    return added;
}

fn genAddLocal(ident: ~>readonly char): bool
{
    assert(inFnDef);
    local added: bool = symtabAddLocal(ident, -8 * (localVarCount + 1));
    ++localVarCount;
    return added;
}

fn genReturn(val: GenReg)
{
    printf("\tmovq \t%%%u, \t16(%%2) \t\t# store return value\n", val);
    printf("\tjmp .leave%d\n", fnDefCount);
}
```

One important detail: Local symbols must not survive beyond their function definition. Add `symtabFreeLocals()` at the end of `genFuncDefEnd()`:

```
fn genFuncDefEnd()
{
    // ... as before
    symtabFreeLocals();
}
```

Test your changes: Rebuild and run `xtest_gen_ex3`. For now, this is only a preliminary test, mainly to verify the stack offsets using `symtabPrintLocals()`. The generated assembly is not yet a complete executable program. We will turn it into one in Worksheet 20. Until then, inspect the generated assembly with the good old eyeball test. ;-)